

گزارش فنی تیم شبیه سازی پاور (POWER)

اعضای گروه : محمد متین قادری، برسام شاملو، سارا نوری، علیرضا استوار، بنیامین خلیلی

استاد راهنما : محدثه رحیمی

پست الکترونیکی :

shamlo1387@gmail.com
matin.kalaigha@gmail.com

چکیده

در این گزارش روند کار و مسیر ما برای شرکت در مسابقات روبوکاپ 2026 توضیح داده می شود. هدف اصلی، معرفی الگوریتم ها و استراتژی هایی است که در برنامه نویسی و شبیه سازی استفاده زبان پایتون به عنوان ابزار اصلی توسعه انتخاب شد. در ابتدا با یادگیری مفاهیم اساسی برنامه نویسی پایتون آغاز کردیم و سپس وارد مرحله طراحی الگوریتم ها و شبیه سازی در محیط webots شدیم. از جمله الگوریتم های پیاده سازی شده می توان به دنبال مختصات مد نظر در زمین گشتن، دنبال کردن توپ پس از پیدا کردن آن، مدیریت موقعیت دروازه بان، و نحوه حرکت در شرایطی که توپ در دید نیست اشاره کرد. در ادامه گزارش، جزئیات بیشتری درباره ساختار برنامه، الگوریتم ها و تجربه های کسب شده در طول مسیر ارائه خواهد شد.

واژه های کلیدی: الگوریتم ها- پایتون- شبیه ساز webots

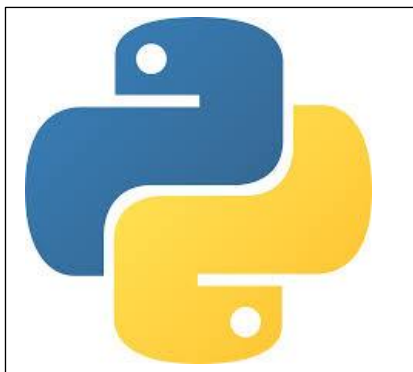
مقدمه

تیم ما در این سال برای اولین بار در لیگ rcj-soccer-sim شرکت میکند با توجه به تجربیات قبلی ذهنیت مناسبی برای مفاهیم پایه برنامه نویسی و حل مسئله های آن فراهم کرده است. علاقه اعضای تیم به لیگ Simulation و همچنین برنامه نویسی از مهم ترین انگیزه های ما برای ورود به مسابقات بوده شرکت در این مسابقات باعث توسعه دانش و تقویت مهارت های برنامه نویسی و همچنین هماهنگی تیم می شود هدف تیم از حضور در این مسابقات کسب تجربه علمی در محیط شبیه سازی برای فعالیت های پیشرفته تر است.

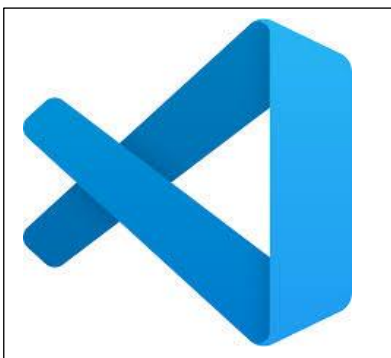
1. نرم افزار :



1.1. **Webots**: در لیگ RCJ Soccer Simulation، مسابقات در محیط شبیه سازی Webots برگزار می شوند. Webots یک شبیه ساز سه بعدی است که امکان مشاهده و بررسی عملکرد روبات ها را در یک زمین فوتبال مجازی فراهم می کند. روبات ها با استفاده از کدهای کنترلی حرکت کرده و با توپ و سایر روبات ها تعامل دارند. این محیط به تیم ها کمک می کند تا الگوریتم ها و استراتژی های خود را قبل از مسابقات واقعی آزمایش و بهینه سازی کنند.



1.2. **Python** : در این پروژه از زبان برنامه نویسی Python برای توسعه و پیاده سازی الگوریتم های کنترلی استفاده می شود. کدهای نوشته شده وظیفه تصمیم گیری، حرکت، و تعامل روبات ها با توپ و سایر بازیکنان را بر عهده دارند. سادگی و انعطاف پذیری پایتون باعث می شود فرآیند توسعه، تست و بهینه سازی رفتار روبات ها سریع تر و دقیق تر انجام شود.



1.3. **Visual studio code**: نرم افزار Visual Studio Code یک محیط توسعه (IDE) قدرتمند و سبک است که برای نوشتن، ویرایش و مدیریت کدهای پروژه از آن استفاده می کنیم. این برنامه با پشتیبانی از زبان پایتون و افزونه های متنوع، فرایند برنامه نویسی، اشکال زدایی و توسعه ربات ها را ساده تر و سریع تر می کند.

2. الگوریتم ها

2.1. پیدا کردن مختصات توپ

این تابع به ربات کمک می کند موقعیت توپ را روی زمین با دقت بیشتری مشخص کند. در ابتدا، فاصله توپ با استفاده از یک فرمول ساده اصلاح می شود تا مقدار به دست آمده از دوربین به مقدار واقعی نزدیک تر شود. سپس زاویه توپ با جهت گیری ربات ترکیب می شود تا مشخص شود توپ نسبت به مختصات کلی زمین در چه جهتی قرار دارد. با انجام محاسبات مثلثاتی، مختصات توپ نسبت به ربات محاسبه شده و در نهایت به موقعیت فعلی ربات اضافه می شود. تا محل نهایی توپ روی زمین به دست آید. به طور خلاصه، این تابع به ربات کمک می کند جهت حرکت مناسب را انتخاب کند و با دقت بیشتری به سمت توپ حرکت کند.

```
def Ball_Coordinates(self):  
    global last_toop_be_zamin_x , last_toop_be_zamin_y , ball_speed , now , delta_time , last_time , ball_angle_predict_deg , newDeg  
    global ballx , bally , ball_dist , ball_angle , ball_angle_predict , distance  
  
    r = 0.11886 * ball_dist - 0.02715  
  
    ball_angle = math.atan2(ball_y , ball_x)  
    ball_angle = math.degrees(ball_angle)  
    if ball_angle < 0:  
        ball_angle += 360  
    theta = (ball_angle+robot_angle+90) % 360  
    if theta < 0:  
        theta += 360  
  
    theta = math.radians(theta)  
    pos_x = r * math.cos(theta)  
    pos_y = r * math.sin(theta)  
    pos_x = round(pos_x, ASHAR)  
    pos_y = round(pos_y, ASHAR)  
  
    ballx = robotx + 2.5 * pos_x  
    bally = roboty + 2.5 * pos_y  
    ballx = round(ballx, ASHAR)  
    bally = round(bally, ASHAR)
```

ball-coordinates

3.2 دور زدن توپ

این منطق مشخص می کند ربات در مواجهه با توپ چگونه تصمیم گیری کند. ابتدا بررسی می شود که توپ نسبت به موقعیت ربات جلوتر است یا عقب تر تا معلوم شود ربات باید وارد حالت چرخش شود یا نه. اگر ربات در حالت عادی باشد، تلاش می کند خودش را در بهترین زاویه نسبت به توپ قرار دهد تا کنترل بیشتری روی آن داشته باشد. در شرایطی که توپ با شدت بیشتری حرکت می کند، ربات با تغییر مسیر یا کمی عقب رفتن سعی می کند تعادل و موقعیت مناسب را حفظ کند. اگر ربات بیش از حد جلو زمین قرار گرفته باشد، به یک موقعیت امن تر برمی گردد تا از اشتباه دفاعی جلوگیری شود. در مجموع، این رفتار باعث می شود ربات هوشمندانه تر، نرم تر و هماهنگ تر با وضعیت توپ حرکت کند.

```
def turn(self):
    global is_turning
    if bally>roboty:
        is_turning=1
    elif AngelBallR>350 or AngelBallR<10:
        is_turning=0

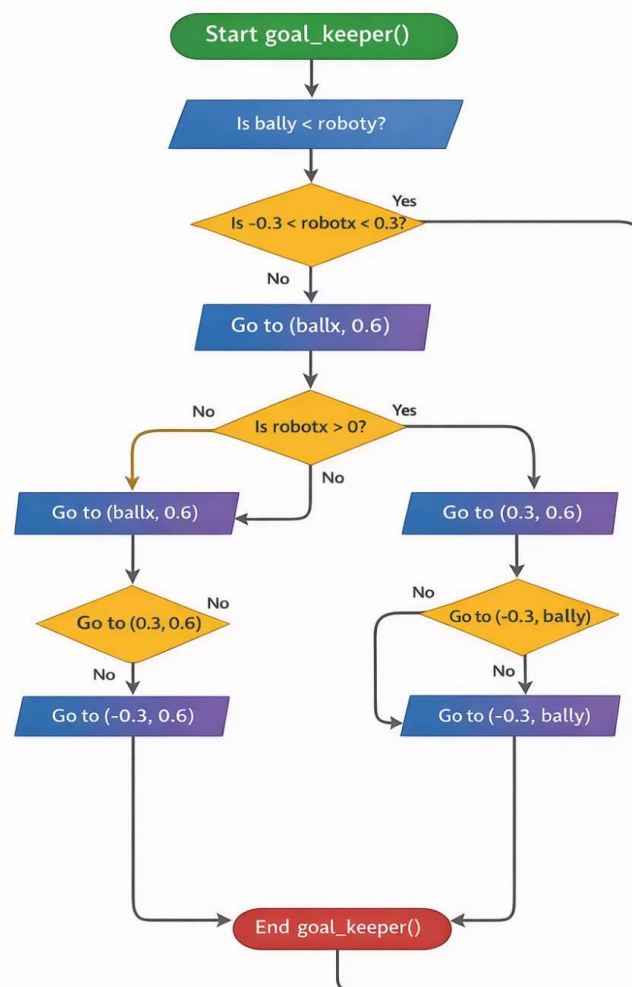
    if is_turning==0 and bally<0.5:
        if 330>AngelBallR>30:
            goToTheta(self,AngelBallR-90)
        else:
            if strength>100:
                go_to(self,0,-0.6)
            else:
                goToTheta(self,AngelBallR-90)

    elif is_turning==1 and bally<0.5:
        if ballx<robotx:
            goToTheta(self,AngelBallR-90 - strength/1.5)
        else:
            goToTheta(self,AngelBallR-90 + strength/1.5)
    elif roboty>0.5:
        go_to(self,0,0.2)
```

تصویری از کد turn

3.3. دروازه بان:

این تابع `goal_keeper` یک رفتار دفاعی ساده برای ربات دروازه بان است. اگر داده‌های سنسورها معتبر باشد و ربات توپ را ببیند، ربات موقعیت توپ و خودش را بررسی می‌کند تا بین توپ و دروازه خودی قرار بگیرد. وقتی ربات توپ را نبیند ربات به یک نقطه امن در مرکز دفاع حرکت می‌کند، اگر توپ عقب‌تر از ربات باشد، ربات به یکی از کناره‌های زمین می‌رود تا مسیر عبور توپ را ببندد. هدف کلی این الگوریتم جلوگیری از نزدیک شدن توپ به دروازه و پوشش دادن فضای دفاعی است.



فلوچارت `goal_keeper`

4. نتیجه گیری :

در فرآیند توسعه سیستم، استراتژی‌ها و الگوریتم‌های متعددی در قالب نسخه‌های مختلف پیاده‌سازی و ارزیابی شدند. در هر مرحله از به‌روزرسانی، عملکرد نسخه‌ی جدید از طریق برگزاری مسابقه با نسخه‌ی قبلی مورد سنجش قرار گرفت و بر اساس نتایج حاصل، بهبودهای لازم اعمال گردید. در نهایت، نسخه‌ی فعلی به‌عنوان حاصل این روند تدریجی انتخاب شد..

رویکرد اصلی تیم، حرکت به سمت پیاده‌سازی راه‌حل‌هایی بهینه، ساخت‌یافته و خلاقانه بوده است؛ به‌گونه‌ای که ضمن افزایش کارایی، پیچیدگی و حجم کد نیز کاهش یابد. همچنین با به‌کارگیری از محاسبات ریاضی در توابع مختلف به حداقل رسیده و انسجام و خوانایی کد بهبود یافته است..

در نهایت، با تحلیل عملکرد تیم در مسابقات آزمایشی، تمرکز بیشتری بر تقویت ساختار دفاعی و محافظت مؤثر از دروازه اعمال شد که تأثیر قابل‌توجهی بر عملکرد کلی سیستم داشته است .

5. نوآوری:

ما یک سیستم کنترلی طراحی کردیم که باعث می‌شود دو ربات مهاجم به‌صورت هماهنگ و هم‌زمان در طول زمین حرکت رفت و برگشتی داشته باشند. این هماهنگی طوری برنامه‌ریزی شده که هر دو ربات مثل یک تیم منسجم عمل کنند و بدون تداخل در مسیر هم، فضای زمین را به‌طور کامل پوشش بدهند.

این حرکت هم‌زمان باعث می‌شود فشار مداوم روی حریف حفظ شود و فرصت‌های بیشتری برای نزدیک شدن به هدف و اجرای حمله ایجاد شود. به زبان ساده، ربات‌ها به‌جای حرکت‌های جداگانه، با یک استراتژی مشترک جلو و عقب می‌روند و مثل دو بازیکن هماهنگ، زمین را تحت کنترل خودشان می‌گیرند

6. تشکر و قدردانی:

تشکر و تقدیر فروان از جناب آقای یوسفی، سرکار خانم لطفی، سرکار خانم رحیمی، و از خانواده‌های محترم مان که در این راه از ما حمایت های فراوانی کردن و در نهایت از داوران عزیز قدردانیم که این فرصت را برای ما به وجود آوردند، بسیار سپاس گزاریم .

7. مراجع و منابع:

- <https://junior.forum.robocup.org/c/robocupjunior-soccer/5>
- <https://github.com/robocup-junior/rcj-soccersim>
- <https://www.python.org/>